LazyCodingAgent — Agent Tools

These are the functions the LazyCodingAgent LLM can call while helping you in the workspace. Each call uses JSON arguments; the app returns JSON results (errors include an error field).

read_file

What it does

Lets the agent inspect project files without executing code. Returns line-numbered text so it can cite locations; use offset and limit to read large files in chunks (up to 2000 lines, 16 MiB cap).

Input

- path (string) — required
- offset (integer) — optional, 1-based start line
- limit (integer) — optional, max lines (default 2000)

Output

- path, content (line-numbered), offset, lines_returned, total_lines
- On failure: error message in JSON

Modes: Ask, Plan, Agent

write_file

What it does

Creates a new file or replaces an entire file in one step. The agent sends the full new content; missing parent folders are created automatically under the workspace.

Input

- path (string) — required
- content (string) — required

Output

- status, path, bytes_written, changed_paths
- Optional: tracking_warning
- On failure: error

Modes: Agent only

list_directory

What it does

Explores the project one directory at a time, like a single folder in a file browser. Lists immediate children (files, folders, symlinks) sorted by name, up to 500 entries per call.

Input

- path (string) — optional, default project root
- include_hidden (boolean) — optional

Output

- directory, entries[] (path, type, size), total_entries, truncated
- On failure: error

Modes: Ask, Plan, Agent

glob

What it does

Finds many matching paths under a folder using wildcards (*, ?, **). Use when the agent knows a naming pattern (e.g. all units under src/) instead of walking directories one level at a time.

Input

- pattern (string) — required, e.g. src/**/*
- path (string) — optional search root
- include_hidden (boolean) — optional

Output

- directory, entries[] (path, type, size), total_entries, truncated
- On failure: error

Modes: Ask, Plan, Agent

search_code

What it does

Searches file contents across the workspace, similar to IDE Find in Files. Supports literal text or regex, optional path and glob filters; skips oversized files, binary data, and returns up to 500 matches.

Input

- query (string) — required
- path (string) — optional file or directory
- glob (string) — optional path filter
- regex, case_sensitive, include_hidden (boolean) — optional

Output

- matches[] (path, line, column, text), total_matches, truncated
- On failure: error

Modes: Ask, Plan, Agent

apply_patch

What it does

Applies multi-line edits using standard unified diff format. The agent validates every hunk before writing; can create, modify, or delete files—preferred for structured code changes.

Input

- patch (string) — required unified diff text

Output

- status, changed_paths
- Optional: warnings
- On failure: error, error_kind, hint; may include partial

Modes: Agent only

edit_file

What it does

Performs one exact search-and-replace in a single file. Best for a small, localized change when a full patch is awkward; old_text must match byte-for-byte exactly once in the file.

Input

- path (string) — required
- old_text (string) — required, exact match once
- new_text (string) — required

Output

- status, changed_paths
- Optional: tracking_warning, warnings
- On failure: error

Modes: Agent only

shell

What it does

Runs terminal commands such as builds, tests, and package installs. Executes under the project root (or optional cwd) with a timeout; captures stdout/stderr and may refresh the workspace file tree after run.

Input

- command (string) — required
- cwd (string) — optional working directory
- timeout_ms (integer) — optional (default 120000)

Output

- exit_code, stdout, stderr, timed_out, cancelled
- stdout_truncated, stderr_truncated, changed_paths, refresh_project
- On failure: error

Modes: Agent only

diagnostics

What it does

Shows compiler and linter messages so the agent can fix build errors. Read returns the last cached result; in Agent mode, build runs an auto-detected or custom command and parses output into structured messages.

Input

- action (string) — required: read or build
- target (string) — optional project/manifest/source
- timeout_ms (integer) — optional for build
- command (string) — optional custom build command

Output

- read: cached diagnostics JSON or status unavailable
- build: status, messages[], runner, target, exit_code, stdout, stderr
- On failure: error

Modes: Ask, Plan, Agent (build action: Agent only)

git

What it does

Read-only view of Git history and working tree changes. The agent can check status, diffs, log, and show commits—without committing, pushing, or other mutating Git operations.

Input

- operation (string) — required: status | diff | log | show
- revision (string) — optional
- paths (array of strings) — optional
- limit (integer) — optional, 1–100 for log
- staged (boolean) — optional for diff

Output

- exit_code, stdout, stderr
- Optional: error if Git failed
- On failure: error

Modes: Ask, Plan, Agent

todo

What it does

Maintains a task checklist for the current chat session only. Helps the agent track what is pending, in progress, or done while working on a multi-step request (not saved to project files).

Input

- action (string) — required: read or replace
- items (array) — required for replace: id, text, status per task

Output

- items[] (id, text, status)
- On failure: error

Modes: Ask, Plan, Agent

create_plan_file

What it does

Saves a finished implementation plan as Markdown in the workspace .plan/ folder. Used in Plan mode so the user can review steps before Agent mode applies code changes; each file gets a unique timestamped name.

Input

- content (string) — required, markdown plan (max 1 MiB)

Output

- status, path, content
- On failure: error

Modes: Plan, Agent

grep

Alias for search_code — same behavior, inputs, and outputs.

What it does

Searches file contents across the workspace, similar to IDE Find in Files. Supports literal text or regex, optional path and glob filters; skips oversized files, binary data, and returns up to 500 matches.

Input

- query (string) — required
- path (string) — optional file or directory
- glob (string) — optional path filter
- regex, case_sensitive, include_hidden (boolean) — optional

Output

- matches[] (path, line, column, text), total_matches, truncated
- On failure: error

Modes: Ask, Plan, Agent

terminal

Alias for shell — same behavior, inputs, and outputs.

What it does

Runs terminal commands such as builds, tests, and package installs. Executes under the project root (or optional cwd) with a timeout; captures stdout/stderr and may refresh the workspace file tree after run.

Input

- command (string) — required
- cwd (string) — optional working directory
- timeout_ms (integer) — optional (default 120000)

Output

- exit_code, stdout, stderr, timed_out, cancelled
- stdout_truncated, stderr_truncated, changed_paths, refresh_project
- On failure: error

Modes: Agent only

run_process

Alias for shell — same behavior, inputs, and outputs.

What it does

Runs terminal commands such as builds, tests, and package installs. Executes under the project root (or optional cwd) with a timeout; captures stdout/stderr and may refresh the workspace file tree after run.

Input

- command (string) — required
- cwd (string) — optional working directory
- timeout_ms (integer) — optional (default 120000)

Output

- exit_code, stdout, stderr, timed_out, cancelled
- stdout_truncated, stderr_truncated, changed_paths, refresh_project
- On failure: error

Modes: Agent only

plan

Alias for todo — same behavior, inputs, and outputs.

What it does

Maintains a task checklist for the current chat session only. Helps the agent track what is pending, in progress, or done while working on a multi-step request (not saved to project files).

Input

- action (string) — required: read or replace
- items (array) — required for replace: id, text, status per task

Output

- items[] (id, text, status)
- On failure: error

Modes: Ask, Plan, Agent